

Let’s Play Proxy War

From a working starter policy to replay-driven improvement

Recommendation

Yes—the CrewRift learning format is the right shape for Proxy War. The hard infrastructure already exists. The opportunity is to connect it into one coherent, executable journey: understand a decision, change a policy, prove the choice is legal, run a match, inspect the replay, and improve one thing.

FOUNDATION	MISSING LAYER	THE ASK
Starter policies, a legal-action contract, packaging and upload tooling, the canonical match runtime, replays, and decision evidence.	A guided experience that makes the whole path visible, runnable, and understandable to someone new to Proxy War.	Adapt the CrewRift play-and-analyze loop as a learning layer over the existing system—not as a new SDK or game implementation.

PURPOSE	Align on the desired notebook experience and the materials already available.
AUDIENCE	A technical learning-experience collaborator familiar with notebooks, containers, and agents; no prior Proxy War context assumed.
STATUS	External-shareable working brief • Stable protocol facts separated from live service details.

PROPOSED / CONTEXT AND RECOMMENDATION

1. The opportunity

The useful part of CrewRift’s “Let’s Play” notebook is its learning loop: contract → working policy → offline examples → packaged agent → real game → replay → one controlled improvement. Proxy War already has each technical piece. What it lacks is one polished path across them.

The notebook should expose and orchestrate the system we have. It should not generate a parallel client, rewrite the protocol in Python, or introduce a second validator or runner.

KEEP	ADAPT	AVOID
Progressive explanation, editable examples, frozen scenarios, a clean run, replay analysis, and a clear next experiment.	Replace CrewRift-specific actions and fixtures with Proxy War observations, offered legal actions, replays, and telemetry.	Hard-coded league IDs, unpinned environments, notebook-only protocol code, hidden fallbacks, or publishing from “Run All.”

REPOSITORY-VERIFIED / PRODUCT AND CONTRACT

2. Proxy War in 60 seconds

Proxy War is an autonomous strategy game in which software agents claim territory, build an economy, form alliances, attack rivals, and play complete matches without human input. An agent—called a policy—runs in its own container. At each decision, it receives the current observation and a list of actions that are legal now.



The one rule that matters

A policy selects one exact LegalAction.id from the list it was offered. It never constructs or sends a raw game-engine intent. An invented or stale ID is rejected by the existing validator.

3. What already exists

The material is substantial. The missing piece is a guided experience across it—not another protocol or starter implementation.

Asset	What it provides	Notebook role
Starter README + onboarding	Public landing page and first-policy walkthrough.	Orient the learner; link out for setup details.
llm-player.mjs	Editable LLM policy with background planning and guarded action selection.	Expose the three safe customization points.
starter-player.mjs	Compact non-LLM baseline.	Supply a deterministic control policy.
launch.sh	Preflight, container build, browser sign-in, upload, and policy-version ID lookup.	Keep upload as an explicit optional terminal step.
Player protocol	Minimal decision request/response contract.	Anchor the contract tour and fixtures; use the starter for full transport behavior.
Adapter certification + episode tooling	Canonical local integration and replay-verification paths.	Invoke behind the notebook; do not reimplement.
Replays + decision evidence	Rendered match evidence plus policy-authored reason summaries, confidence, and degradation signals.	Power the analyze-and-improve chapter from a reviewed bundle or authorized view.

Two layers, one architecture

The lightweight external starter is the learner-facing policy. The Coworld adapter is the engine/integration layer. The notebook may connect them, but it must keep their responsibilities distinct.

4. How the supplied LLM starter works

The starter separates slow strategic reasoning from time-sensitive move selection. A model writes a compact standing plan in the background; the decision handler immediately maps the latest good plan to one currently offered legal action.

Builder edits	Purpose	Boundary
STRATEGY	Plain-language standing orders: how the nation should play.	Changes goals, not the transport contract.
buildState	Selects the game facts shown to the planner.	Uses the observation; does not bypass it.

Builder edits	Purpose	Boundary
choose	Maps the standing plan to one offered action.	Must return an item from legalActions.

The starter also keeps anti-loop memory, sanitizes untrusted game text, continues from the last good plan when planning fails, and emits fallbackUsed and llmPlannerDegraded so a strategic loss can be distinguished from an unhealthy planner.

REPOSITORY-VERIFIED / CURRENT WORKFLOW

5. The builder journey today

1. **Check the environment.** Run `bash launch.sh --doctor` for a non-mutating preflight.
2. **Choose a starting point.** Use the LLM policy or the compact rule-based baseline.
3. **Customize the policy.** Edit STRATEGY, buildState, and choose while preserving the transport and legal-action contract.
4. **Validate through canonical tooling.** Exercise fixtures and the existing adapter's certification or verified episode path.
5. **Build and upload deliberately.** The launcher can package and upload a policy after browser authentication.
6. **Seat or submit the policy.** A separate invited-match or current league step is required.
7. **Inspect and improve.** Use the result, rendered replay, decision reasons, confidence, and degradation telemetry.

Important correction to the current onboarding language

`bash launch.sh my-agent` builds and uploads a policy version. It does not itself run a match. That version must then be seated in an invited episode or submitted to a live league.

The current starter commands

```
git clone https://github.com/0xNad/ProxyWar.git
cd ProxyWar/coworld-adapter/tester-starter-llm

# Non-mutating environment check
bash launch.sh --doctor

# Explicit authenticated build + upload
bash launch.sh my-agent
```

Hosted authentication, model access, league identifiers, and current CLI compatibility are live-service facts. Recheck them at the time of use; do not freeze them into the durable notebook.

6. Desired notebook experience

Present this as one journey with two linked chapters. They may live in one notebook or two, but the learner should experience a continuous loop from a working decision to a measured improvement.

CHAPTER A Let's Play Proxy War • Preview a rendered replay before setup. • Inspect one frozen decision request. • Run the rule baseline on that fixture. • Edit strategy, state selection, or choice logic. • Prove the selected ID was offered. • Invoke a canonical episode and verify its replay. • Open the evidence produced by the match.	CHAPTER B Let's Analyze & Improve • Read the result and territory trajectory. • Summarize action mix and decision reasons. • Separate strategy failure from fallback or degradation. • Form one explicit hypothesis. • Change one behavior—not five at once. • Rerun the same controlled scenario. • Compare baseline and challenger evidence.
---	--

Safe “Run All” behavior

The core notebook path must be deterministic and non-publishing. Upload, league submission, browser sign-in, and any other hosted mutation belong in a clearly separated optional final section that requires explicit user action.

7. Guardrails

Do	Do not
Invoke the existing Node policy and canonical adapter paths.	Rewrite the protocol or policy runtime as notebook-only Python.
Choose exactly one supplied LegalAction.id.	Construct raw OpenFront intents or bypass validation.
Use frozen, sanitized fixtures for offline teaching.	Depend on a changing hosted league for the core lesson.
Expose fallback and planner-degradation telemetry.	Hide an unhealthy strategic brain behind legal executor output.
Fail loudly with a useful next step.	Save secrets, private paths, or raw internal logs in outputs.

8. Deliverables

- A guided “Let’s Play Proxy War” chapter and a linked “Let’s Analyze & Improve” chapter; one notebook or two is acceptable.
- A small frozen and sanitized evidence bundle: representative decision requests, one result, one verified replay, and decision metadata.
- A minimal helper layer that calls canonical policy and adapter code instead of duplicating it.
- A concise README and a pinned or recorded tested environment.
- Executed HTML or PDF exports with meaningful outputs, plus an automated clean-run check.
- Existing licenses and attribution retained for every reused component.

9. Definition of done

- ☐ A fresh macOS, Linux, or WSL user can run the core path top to bottom without internal-repository access or hosted credentials.
- ☐ Every saved executable cell has an execution count and meaningful output; a clean-room or CI run proves it.
- ☐ The selected action is always one of the supplied legal IDs, and an invalid-ID fixture visibly fails.
- ☐ A full episode uses the canonical adapter, validator, and runner; the resulting replay verifies.
- ☐ Analysis shows the outcome, action mix, decision reasons, and fallback/planner-degradation signals.
- ☐ The learner completes one reproducible baseline-versus-change comparison.
- ☐ “Run All” cannot upload, submit, publish, or trigger browser authentication.
- ☐ No secrets, private paths, internal project-state files, or unreviewed raw logs are included.
- ☐ Errors fail loudly and point to the next concrete action.

Success in one sentence

A successful first version takes someone with no Proxy War context from a clean checkout to a validated agent decision and a replay-based improvement experiment, with every step executed, inspectable, and reproducible.

10. Public-safe handoff

Share these curated public links. Do not send the whole development workspace: it adds internal operating material without helping the collaborator understand the agent path.

Public material	Link and use
Current Proxy War source	github.com/OxNad/ProxyWar ↗ Moving public landing page for the current codebase.
Reviewed starter snapshot	Open immutable reviewed snapshot ↗ Immutable technical source used for this brief. Treat hosted-runtime language inside it as dated and recheck it.
Reviewed protocol snapshot	Open immutable reviewed snapshot ↗ Immutable minimal decision contract; the starter shows the wider transport lifecycle.
Coworld source	github.com/Metta-AI/coworld ↗ The upstream packaging and league tooling.
Public league view	beta.proxywar.xyz/league ↗ Live demonstration; treat current contents as changeable, not source truth.
Softmax Observatory	softmax.com/observatory ↗ Hosted policy and match evidence; Softmax sign-in is currently required.

Materials still to provide for the prototype

- One scrubbed decision_request fixture that exercises a meaningful choice.
- One reviewed replay/results bundle with decision metadata and no private logs.
- One stable replay screenshot or public-safe preview asset.
- The exact environment versions proven by the clean-run check.

Source synchronization note

At the time of this audit, the separate proxywar-coworld-starter repository trails the newer starter inside the main ProxyWar repository. Use the reviewed technical snapshot above until the standalone repository is synchronized, and recheck any hosted-service claims inside it.

Keep out of the handoff

Internal project-state, operating-layer files, competitive strategy documents, raw model prompts or logs, temporary certification artifacts, credentials, authentication material, and operator runbooks.

11. Recommended prototype

Prove the learning loop before the full hosted path

Frozen request → starter decision → canonical legal-ID validation → supplied replay/results bundle → one diagnostic chart. Once that is clean, add a full local episode and keep hosted upload/submission as an explicit optional step.

This is deliberately smaller than a complete end-to-end integration. It proves the collaborator can teach the right architecture and create a useful analysis loop before either side spends time on live-service edge cases.

Working split for the first prototype

Proxy War provides	Notebook collaborator provides
Reviewed source and protocol snapshots.	An executed, progressive notebook structure.
A scrubbed decision-request fixture.	A minimal helper layer and clear visual outputs.
A reviewed replay/results evidence bundle.	One baseline-versus-change analysis experiment.
Canonical validation hooks and acceptance review.	Recorded versions, clean-run proof, and an export.

Decision requested

Confirm whether this play → analyze → improve shape matches the collaboration you had in mind. If it does, the next step is to exchange the scrubbed fixture and evidence bundle and build the deliberately small prototype above.